[TOPIC 20] 4강. GAN 알아보기

※ 최종 업데이트: 2026. 06. 21.

지난 시간에 VAE 로 첫 생성 모델을 만들어 봤어요. 입력을 분포로 압축하고, 그 분포에서 뽑은 점을 디코더로 펼쳐 새 이미지를 얻는 구조였죠. 그런데 결과물들이 어딘가 뭉개진 느낌이 있었습니다. 윤곽은 맞는데 또렷하지가 않은, 살짝 흐릿한 결과였어요.

이번 시간에는 색다른 발상으로 생성에 접근하는 **GAN(Generative Adversarial Network)**을 살펴봅니다. VAE 가 분포를 직접 계산하며 한 모델 안에서 압축과 복원을 했다면, GAN 은 두 신경망을 맞붙여 경쟁시켜요. 한쪽은 가짜를 만들고, 다른 한쪽은 그게 가짜인지 가려내죠. 이 단순한 경쟁이 어떻게 VAE 보다 또렷한 이미지를 만들어 내는지, 그리고 그 대가로 무엇이 까다로워지는지 함께 따라가 봅시다.

이번 강의에서는 아래 내용에 대해 함께 알아보겠습니다.

4-1. VAE 에서 GAN 으로

4-2. 생성자와 판별자

4-3. 경쟁이라는 학습

4-4. GAN 의 한계점

4-1. VAE 에서 GAN 으로

VAE 의 흐릿함은 학습에 쓰는 손실에서 비롯돼요. VAE 는 원본과 복원을 픽셀 단위로 비교하는 **재구성 손실(MSE, 평균제곱오차)**로 학습하는데, 이 손실은 한 입력에 대응하는 그럴듯한 답이 여럿일 때 한계를 드러냅니다. 예를 들어 손으로 쓴 3 은 꼬리의 길이나 기울기가 사람마다 다른데, 이 여러 가능성 중 하나를 또렷하게 선택했다가 정답과 어긋나면 픽셀 오차가 크게 벌어져요. 반대로 여러 가능성을 평균 낸 형태를 그리면 어떤 정답과도 크게 어긋나지 않아 오차가 가장 작게 유지되죠. 그래서 재구성 손실만으로 학습한 모델은 또렷한 한 형태 대신 평균적인 형태를 선호하게 되고, 그 평균이 곧 뭉개짐으로 나타납니다.



MSE Loss 로 학습하면 결과물이 흐릿하게 생성되는 경향을 띈다.

GAN 은 이 손실의 기준 자체를 바꿔요. 픽셀이 원본과 얼마나 일치하는지를 재는 대신, **생성된 이미지가 진짜 데이터처럼 보이는지를 판단**하는 별도의 신경망을 둡니다. 그리고 이미지를 생성하는 신경망은 그 판단을 통과하도록, 곧 진짜로 분류되도록 학습해요. 평균을 낸 흐릿한 이미지는 진짜 데이터에서 거의 나타나지 않으므로 가짜로 분류되고, 따라서 평균적인 형태로는 목표를 이룰 수 없습니다. 진짜 데이터와 구별되지 않을 만큼 또렷한 이미지를 만들어야만 하죠. 손실의 기준이 픽셀 일치에서 **진짜다움으로 옮겨 간 것**이 GAN 의 출발점입니다.

여기에는 VAE 와의 더 근본적인 차이가 있어요. VAE 는 잠재 분포를 표준 정규분포에 맞추는 방식으로 데이터의 분포를 명시적으로 다뤘습니다. 반면 GAN 은 데이터가 나올 확률을 직접 계산하지 않아요. 무엇이 진짜에 가까운지는 판단하는 신경망이 알려주고, 생성하는 신경망은 그 판단 결과에만 의존하여 학습됩니다.



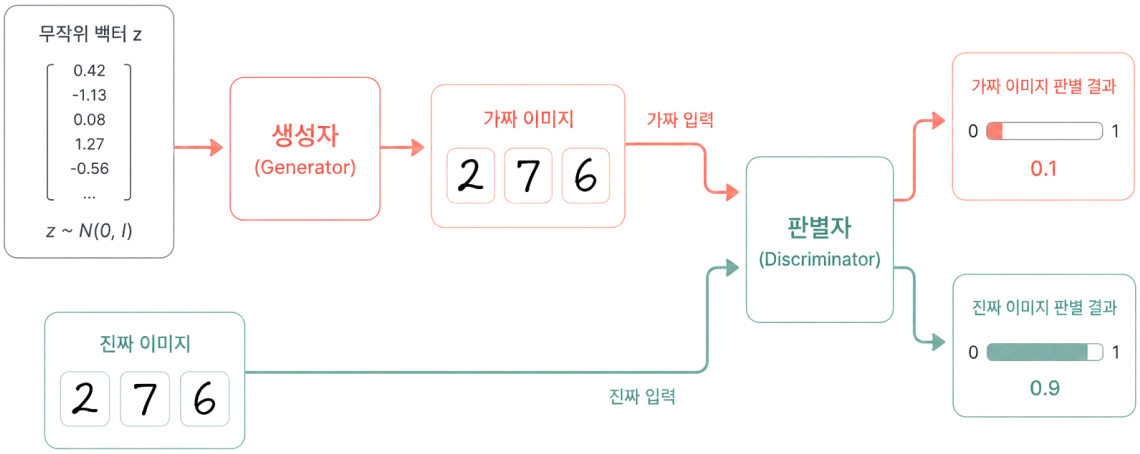
VAE 와 GAN 의 학습 결과물에 대한 차이

4-2. 생성자와 판별자

GAN 은 이름 그대로 두 신경망이 **적대적으로 맞서는(Adversarial) 구조**예요. 자주 사용되는 예시가 위조지폐를 만드는 위조범과 그것을 가려내는 감별사입니다.

생성자(Generator) 는 위조범이에요. 아무 의미 없는 무작위 잡음 z 를 입력으로 받아, 그것을 그럴듯한 가짜 이미지로 펼쳐 냅니다. 입력이 무작위 값이라는 점, 그리고 거기서 이미지를 만들어 낸다는 점에서 생성자는 VAE 의 디코더와 똑 닮았어요. 다른 점은 생성자에게는 "이 입력을 이렇게 복원하라" 고 짝지어 주는 정답 이미지가 없다는 거예요. VAE 의 디코더는 원본이라는 정답을 보고 따라 그렸지만, 생성자는 정답 없이 오직 진짜처럼 보이게만 만들라는 주문만 받습니다.

판별자(Discriminator) 는 감별사예요. 이미지를 한 장 받아서, 그것이 진짜 데이터인지 생성자가 만든 가짜인지를 판단해 0 과 1 사이의 점수 하나로 내놓습니다. 진짜라고 확신하면 1 에 가깝게, 가짜라고 의심하면 0 에 가깝게 매기는 식이에요. 말하자면 "이 이미지가 진짜일 확률은 이 정도다" 라고 답하는 분류기인 셈이죠. 진짜 사진엔 0.9, 어설픈 가짜엔 0.1 을 주는 식으로요.



GAN 을 구성하는 두 신경망, 생성자와 판별자의 동작 방식

두 신경망은 정면으로 부딪힙니다. 생성자는 판별자가 자기 가짜에 높은 점수를 주도록, 즉 진짜로 착각하도록 만들고 싶어 해요. 판별자는 반대로 가짜엔 끝까지 낮은 점수를 주며 속지 않으려 하고요. 한쪽이 이기면 다른 쪽이 지는 구도라, 둘은 같은 판 위에서 서로 반대 방향으로 잡아당기게 됩니다. 이 팽팽한 대립 구도를 활용해서 GAN 을 학습하는 거예요.

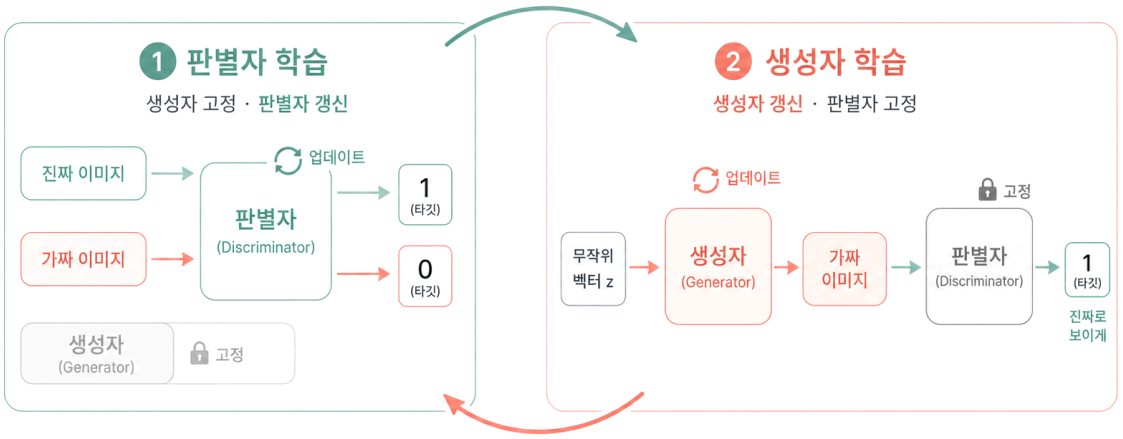
4-3. 경쟁이라는 학습

이제 두 신경망을 실제로 어떻게 학습시키는지 봅시다. 핵심은 둘을 한꺼번에 학습시키지 않고, 한쪽의 가중치를 고정한 채 다른 한쪽만 번갈아 학습시킨다는 점이에요.

먼저 **판별자**를 학습시킵니다. 판별자는 입력 이미지를 진짜와 가짜 두 분류로 나누는 **이진 분류기(Binary Classifier)** 예요. 진짜 데이터에는 1, 생성자가 만든 가짜에는 0 이라는 라벨을 붙이고, 판별자가 입력에 대해 진짜일 확률을 출력하도록 이진 분류를 학습시킵니다. 진짜는 1 에 가깝게, 가짜는 0 에 가깝게 맞히는 방향으로 판별자의 가중치를 갱신하는 거죠. **이 단계에서 생성자의 가중치는 건드리지 않고 고정**해 둬요. 판별자가 진짜와 가짜를 더 정확히 구분하도록 만드는 단계입니다.

그다음 **생성자**를 학습시킵니다. 이번엔 반대로 판별자의 가중치를 고정한 채, 생성자가 만든 가짜를 판별자에 통과시켜요. 그리고 판별자가 그 가짜를 진짜(1)로 분류하도록, 즉 진짜일 확률을 높게 출력하도록 생성자의 가중치를 갱신합니다. 여기서 **판별자는 고정된 평가 함수 역할**만 해요. 판별자의 출력이 곧 생성자에게 주어지는 학습 신호가 되고, 생성자는 그 신호를 따라 자신의 출력을 진짜에 가깝게 바꿔 갑니다. 정답 이미지가 없어도 학습이 되는 이유가 여기 있어요. 픽셀 단위 정답 대신, 판별자의 분류 결과가 생성자를 이끄는 기준이 되어 줍니다.

이 판별자 한 번, 생성자 한 번이 학습의 한 스텝이에요. 관례적으로는 매 스텝, 즉 미니배치 단위로 판별자를 먼저 갱신하고 곧바로 생성자를 갱신하는 식으로 잘게 번갈아 갱신합니다. 이렇게 한 스텝씩 번갈아 데이터를 한 바퀴 돌면 한 에폭이 지나가요.



GAN 의 학습 프로세스 모식도

한쪽씩 번갈아 학습시키는 이유는, **두 신경망의 목표가 정반대이기 때문**입니다. 판별자는 가짜를 0 으로 분류하려 하고, 생성자는 같은 가짜가 1 로 분류되도록 만들려 합니다. 하나의 기준을 두 신경망이 서로 반대 방향으로 끌기 때문에, 둘을 동시에 갱신하면 **학습이 안정적으로 진행되지 않아요.** 그래서 판별자를 한 번 갱신하고 생성자를 한 번 갱신하는 식으로 번갈아 진행하며 균형을 잡습니다. 이 과정을 반복하면 판별자가 정교해질수록 생성자는 더 정밀한 가짜를 만들어야 하고, 생성자가 정밀해질수록 판별자는 더 미세한 차이를 잡아내야 해요.

이 경쟁은 멈추는 이상적인 지점은 **생성자가 진짜와 구분되지 않는 가짜를 만들어 내는 순간**이에요. 일단은 우리는 **뛰어난 생성 모델을 만들고 싶다는 목적**이 있으니까요. 이때 판별자는 어떤 입력에 대해서도 진짜인지 가짜인지 판단할 근거를 찾지 못하고, 진짜일 확률을 0.5 부근으로 출력하게 됩니다. 두 부류를 더는 구분할 수 없다는 뜻이죠. 학습이 이 정도로 진행되면 판별자는 떼어 내고, **무작위 잡음을 생성자에 입력해 새 이미지를 생성하는 데 사용**합니다. 판별자는 생성자를 학습시키기 위한 장치였고, 학습이 끝난 뒤 실제 생성에 쓰는 것은 결국 생성자인 것이죠.

지금까지 말로 설명한 두 목표를 손실 함수로 적어 볼게요. 판별자를 D, 생성자를 G 라고 쓰겠습니다. $D(x)$ 는 판별자가 이미지 x 를 진짜라고 본 확률이고, $G(z)$ 는 생성자가 잡음 z 로 만들어 낸 가짜 이미지예요.

판별자는 진짜 x 에는 1, 가짜 $G(z)$ 에는 0 을 출력하려 합니다. 라벨이 진짜는 1, 가짜는 0 인 이진 분류와 똑같으니, 손실도 이진 분류에서 쓰던 것을 그대로 가져와요. 정답 라벨 y 와 모델의 예측 확률 $\hat{y}$ 에 대한 **이진 교차 엔트로피(Binary Cross Entropy)** 는 이렇게 생겼었죠.

$$BCE = -\left[y \log \hat{y} + (1-y) \log (1-\hat{y})\right]$$

판별자 학습은 이 식을 진짜와 가짜 두 경우에 적용한 것과 똑같아요. 예측 확률 $\hat{y}$ 자리에 판별자의 출력이 들어가는데, 진짜일 때는 $y=1, \hat{y}=D(x)$ 라서 $-\log D(x)$ 만 남고, 가짜일 때는 $y=0, \hat{y}=D(G(z))$ 라서 $-\log(1-D(G(z)))$ 만 남죠. 둘을 합치면 그대로 판별자의 손실이 됩니다.

$$Loss_D = -\left[\log D(x) + \log\left(1 - D(G(z))\right)\right]$$

$\log D(x)$ 항은 진짜를 진짜로 맞힐수록, 즉 $D(x)$ 가 1 에 가까울수록 커지고, $\log(1 - D(G(z)))$ 항은 가짜를 가짜로 맞힐수록, 즉 $D(G(z))$ 가 0 에 가까울수록 커집니다. 앞에 음수 부호가 붙어 있으니, 이 $Loss_D$ 를 줄이는 것은 곧 진짜와 가짜를 모두 잘 맞히는 방향이에요.

생성자는 반대로 자기 가짜가 진짜(1)로 분류되기를 바랍니다. $D(G(z))$ 가 1 에 가까워지길 원하므로, 생성자의 손실은 이렇게 적어요. 이 역시 가짜의 라벨을 1 로 둔 이진 교차 엔트로피, 곧 위 BCE 에서 $y = 1$, $\hat{y} = D(G(z))$ 를 넣은 형태예요.

$$Loss_G = -\log D(G(z))$$

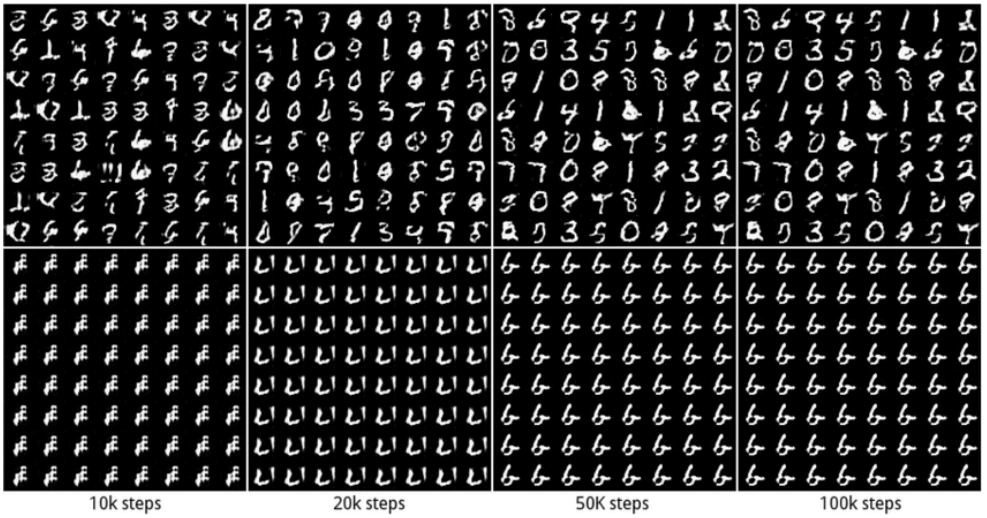
$D(G(z))$ 가 1 에 가까울수록 $Loss_G$ 가 작아지니, 이 손실을 줄이는 것은 판별자를 속이는 방향입니다. 같은 $D(G(z))$ 를 판별자는 0 으로, 생성자는 1 로 끌고 가려 한다는 점에서 두 손실이 정확히 반대를 향한다는 사실도 식에서 드러나죠.

4-4. GAN 의 한계점

GAN 의 또렷한 결과물은 공짜가 아니에요. 경쟁이라는 방식 자체가 학습을 까다롭게 만들거든요. 가장 큰 골칫거리는 **두 신경망의 힘이 균형을 잃는 경우**예요.

판별자가 생성자보다 지나치게 앞서가면, 판별자는 어떤 가짜든 단번에 알아채고 0 에 가까운 점수만 줍니다. 그러면 **생성자는 무엇을 어떻게 고쳐야 점수가 오를지에 대한 단서를 거의 못 받아요.** 전부 똑같이 형편없다는 평가만 돌아오니, 개선 방향을 잡지 못하고 제자리를 맴돌죠. 반대로 생성자가 너무 앞서가면 판별자가 제 역할을 못 해, **엉성한 가짜에도 후한 점수**를 주게 됩니다. 그래서 GAN 학습은 두 신경망이 비슷한 속도로 함께 강해지도록 균형을 맞추는 게 늘 관건이에요.

또 하나 잘 알려진 문제가 **모드 붕괴(Mode Collapse)** 예요. 생성자가 판별자를 속이는 데 성공한 몇 가지 안전한 이미지만 반복해서 만들어 내는 현상입니다. 예를 들어 0 부터 9 까지 다양한 숫자를 만들어야 하는 생성자가, 유독 잘 통하는 1 한 종류만 계속 찍어 내는 식이에요. 생성자의 목표는 어디까지나 **판별자를 속이는 것이지 다양하게 만드는 것이 아니거든요.** 한 가지 이미지로 판별자를 잘 속일 수 있다면, 굳이 위험을 무릅쓰고 다른 모습을 시도할 이유가 없어요. 그래서 다양성을 포기하고 가장 잘 통하는 한곳에 안주해 버립니다.



정상적으로 학습되는 과정(위)와 모드 붕괴(Mode Collapse) 가 발생한 경우(아래)

이런 불안정함 때문에, GAN 을 안정적으로 학습시키는 여러 기법이 등장 이후로 꾸준히 연구되어 왔어요. 우리가 다음 시간에 다룰 내용도 그 흐름 위에 있습니다.

오늘은 경쟁이라는 발상으로 이미지를 만드는 GAN 을 살펴봤어요. 주요 내용을 다시 정리해 보겠습니다.

- **생성자와 판별자 —** 생성자는 무작위 잡음에서 가짜 이미지를 만들고, 판별자는 그 이미지가 진짜인지 가짜인지 가려내는 이진 분류기임.
- **경쟁 학습 —** 판별자와 생성자를 번갈아 갱신하며 경쟁시키면, 판별자의 분류 결과가 정답을 대신하는 학습 신호가 되어 생성자가 점점 진짜 같은 이미지를 만들게 됨.
- **또렷함과 한계 —** 픽셀 일치가 아니라 진짜다움을 직접 압박해 결과가 또렷하지만, 두 신경망의 균형이 무너지기 쉽고 모드 붕괴 같은 문제가 따름.

지금까지의 GAN 은 무작위로 이미지를 만들 뿐, 무엇을 만들지는 고를 수 없었어요. “숫자 7 을 만들어 줘” 처럼 원하는 종류를 지정하려면 조건을 하나 더해야 합니다. 다음 시간에는 여기에 조건을 붙인 **cGAN** 을 코드로 직접 만들어 보면서, 원하는 종류의 이미지를 생성해 볼 거예요. 다음 시간에 뵙겠습니다.